Ashley

The Wayne State grid and Claude Code, from one wizard. Manual, version 1.0.0

2026-09-27

- Why
- Getting started
- How it works
- What you can ask, and what you cannot
- Installing and building
- Troubleshooting
- Appendix

> Ashley is a VS Code extension for medical students in OpenSpineConsortium projects. It sets up the laptop for the Wayne State HPC grid, keeps a Claude Code worker running there on the student's own subscription, and lets the Claude Code extension on the laptop hand work to it. This manual is the pitch, the first hour, how it works, what you can and cannot ask, installation, and troubleshooting. The design notes and the research behind them are in the repository (`docs/ARCHITECTURE.md`, `docs/RESEARCH.md`).

Why

For Wayne State medical students joining an OpenSpineConsortium project.

You know things no computer scientist on the grid knows. You know why a sacral slope of 9° is implausible, what a transitional vertebra does to a surgeon's count, which patients get the second operation. What you may not have is the year it takes to turn that knowledge into a measurement, a job on 802 CT scans, a figure and a paragraph in a paper. Ashley is built to remove that year.

What Wayne State already gave you

The university runs a grid with more than four hundred compute nodes, NVIDIA H200, L40S and A30 GPUs, hundreds of CPU cores you can hold at once, four terabytes of personal storage and twenty more on the lab's shared volume, and an internet connection from every compute node. Every student with an AccessID may apply for an account. Almost none do, because the entry fee is a week of learning SSH, Slurm, containers and shell quoting, and one misstep runs your job on the login node and gets it killed.

Ashley pays that fee for you. One wizard: your AccessID, one password and one Authenticator code, your GitHub and Hugging Face tokens, and a click in the browser that lets Claude work on the grid as you. From then on the grid never asks you for anything.

What you get

A Claude that lives where the data is. Ashley starts a Slurm job, on a CPU node or one with a GPU, that runs Claude Code inside a container in your grid space, with your own subscription. You talk to it from the Claude Code panel in VS Code, or from a prompt box, or from a phone over ssh if you like. You say *"in my project folder, measure the bi-iliac width on the ten development cases and show me the numbers with units"* and it

reads the manifest, writes the script, runs it, looks at the output, fixes what is wrong and answers. Close the laptop. It is still running.

A conversation that does not forget. Sessions are named. *"In session `thesis`, what did we decide about the exclusion list?"* works next week, from a different node.

Your files, in your editor. One command opens your grid home in VS Code on the compute node where your job runs: Explorer, editor, terminal, and a click on a PNG shows the figure. No X server, no copying files back and forth.

Two Claudes that work together. The Claude Code extension on your laptop gets a handful of tools for the grid. It can plan, and hand the heavy step to the grid worker, and bring the answer back into the panel. You can say *"check the grid, run the pelvic width demo on ten records, and tell me whether the sex difference matches the published value"* in one sentence.

Honesty about limits. When your subscription window is used up, the worker says so, tells you when it resets, and waits. It never uses anyone else's account, never types a token on a command line, and never runs anything heavy on the login node.

What becomes possible

- A first-year student with an idea on Monday has ten measured cases by Tuesday and a pre-registered plan by Friday, because the parts that used to take a month (environment, data path, job script, first plot) now take an afternoon.
- A lead runs three projects at once: each has its own worker, its own sessions, its own journal written by the worker at the end of every task.
- A literature question becomes a scoping review pipeline: the worker searches PubMed and OpenAlex with your criteria, logs every inclusion and exclusion, and drafts the PRISMA table, overnight, on a node you are not paying for.
- A model that needs a GPU trains on an H200 while you are in clinic; the loss curve is a PNG in your grid folder when you come back.

What it asks of you

Follow the recipe. Write the question before the number. Freeze the pre-registration. Let the worker run scripts and jobs, and read what it produced yourself: render the mask, check the spread against the literature, ask what could make a plausible number wrong. Ashley makes the machine easy; the judgement is still the point, and it is yours.

Getting started

Install the extension, run **Ashley: Set up my research environment**, and start a worker. The manual walks through every step with screenshots of what you will see. Your lead can add you to a project card on the board the same week.

Getting started

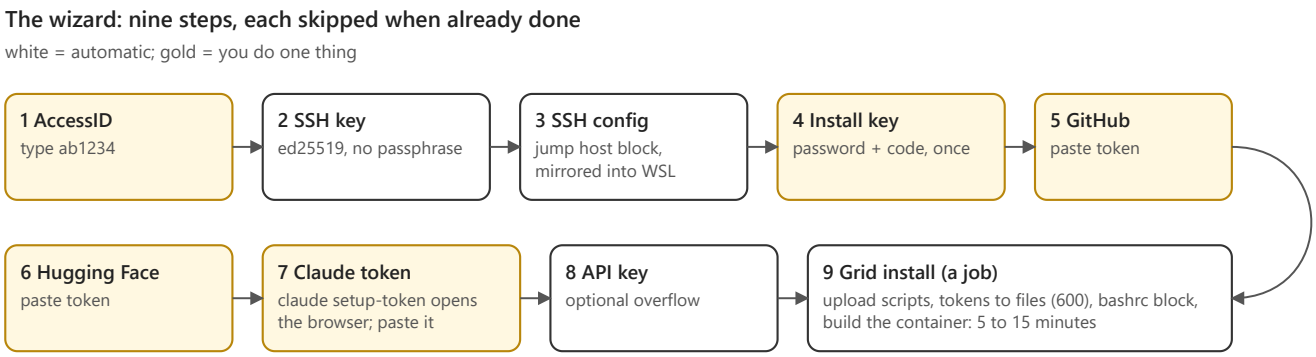
Before you open VS Code

1. **A grid account.** Apply with your AccessID at tech.wayne.edu/kb/high-performance-computing (“Grid account request”), naming your project’s lead as sponsor. Two business days.
2. **Google Authenticator** for the grid (not Microsoft Authenticator): connect to the WSU VPN, log in to ondemand.grid.wayne.edu, Interactive Apps, 2FA Setup, scan the QR code.
3. **A Claude Pro or Max subscription**, and the Claude Code extension signed in.
4. GitHub and Hugging Face accounts.

Install

VS Code, then the extensions **Claude Code**, **Remote - SSH**, and `ashley-wsu-1.0.0.vsix` (Extensions view, `...`, *Install from VSIX*). Ashley offers to run its wizard on first start; or press Ctrl/Cmd+Shift+P and type **Ashley: Set up my research environment**.

The wizard, step by step



The nine wizard steps

Step	What you see	What you do
1 AccessID	an input box	type it (`ab1234`)
2 SSH key	nothing (a second)	nothing
3 SSH config	nothing (a second)	nothing
4 Install key	a terminal titled *Ashley: install key on the grid*	type your grid password, then the six-digit code. Once.
5 GitHub	a dialog, then the browser on the token page (scopes pre-filled)	Generate token, copy it, paste into the input box
6 Hugging Face	same	same
7 Claude token	a terminal running `claude setup-token` ; the browser opens	approve; copy the printed `sk-ant-oat...` token; paste it
8 API key	an input box	leave empty (Enter) unless your lead gave you a key
9 Grid install	a progress note; then "bootstrap job N submitted"	wait 5 to 15 minutes; the Ashley view turns to *Grid ready*

The wizard writes only: a key pair and one block in `~/.ssh/config` on the laptop; tokens in the OS keychain; on the grid `~/.ashley/` , a block in `~/.bashrc` , and a container in your storage. It can be rerun at any time; finished steps are skipped.

Your first worker and your first question

1. **Ashley: Start a Claude worker on the grid**: name `default` , CPU node, 24 hours. A Slurm job is submitted. When it starts, the Ashley view shows `Worker default  <node>  idle` .
2. **Ashley: Ask the grid Claude**: type *"Create a folder hello in my home with a README that says which node you are on and what Python version you have."* The answer streams into an output channel; tool calls appear as italics.
3. **Ashley: Open my grid files on a compute node**: a new VS Code window opens on the node; you see `hello/README.md` in the Explorer.

Talking to it from Claude Code

Ashley: Connect the Claude Code extension to the grid registers the bridge (once). Then, in the Claude Code panel: *"Check the grid status and tell me whether my worker is idle."* The local Claude calls `ashley_status` . For work: *"Ask the grid Claude, in session `demo` , to run `examples/pelvic_width_dimorphism/measure_pelvic_width.py` on ten records and report the mean."*

Every day after that

- Start a worker when you sit down (or keep one running for a day); it costs nothing.
- Name your sessions after the project; a session remembers its folder.
- Let the worker write scripts and Slurm job files; you read them; you say “submit”.
- Pull small results back with git or `rsync`; open PNGs on the node with Remote-SSH.
- If something is off: **Ashley: Check everything (doctor)**. Read the last lines; they name the step and the fix.

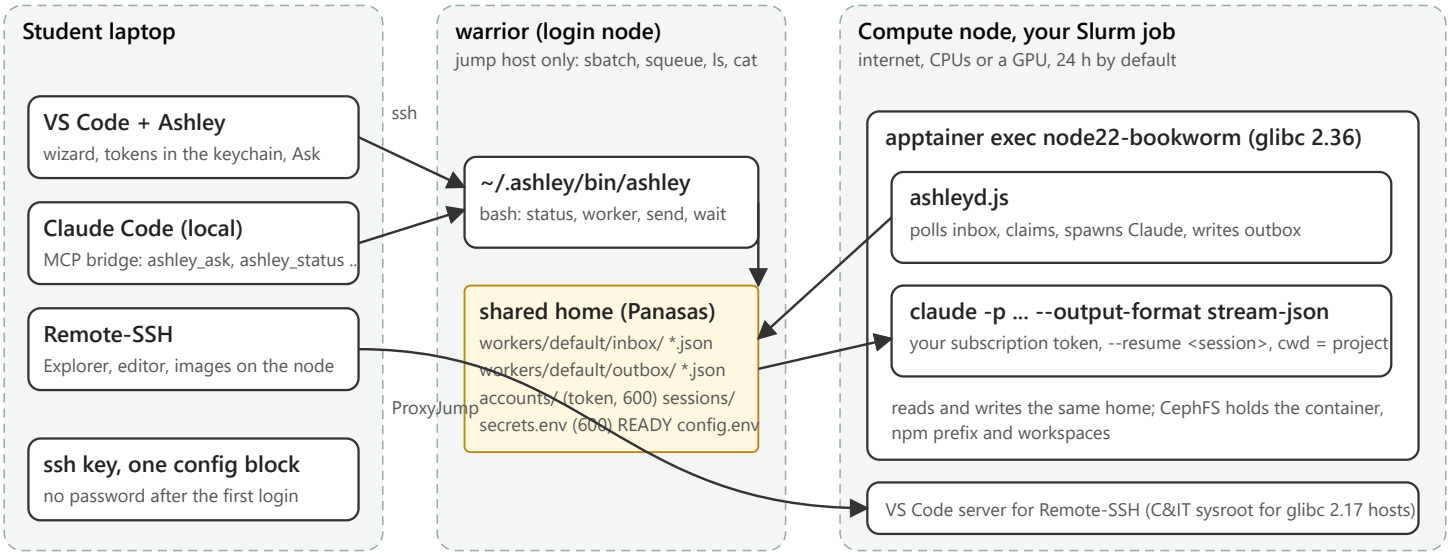
How it works

Ashley is three small things that agree on one file format.

Piece	Runs on	Language	Job
VS Code extension (`extension/`)	the student's laptop, inside VS Code	TypeScript	wizard, secrets, Remote-SSH, the Ashley view, "Ask", MCP registration
MCP server (`extension/dist/mcp-server.js`)	the laptop, spawned by Claude Code	TypeScript (bundled)	gives the local Claude the `ashley_*` tools
Grid CLI + daemon (`grid/`)	the grid: `ashley` on the login node, `ashleyd.js` inside a container on a compute node	bash, Node	install, workers, the request queue, Claude Code itself

Everything the laptop does on the grid is `ssh wsu-grid ~/.ashley/bin/ashley <verb>`. There is no port, no tunnel and no server on the laptop; nothing on the login node beyond `sbatch`, `squeue`, `cat`, `ls` and a half-second sleep loop.

Topology



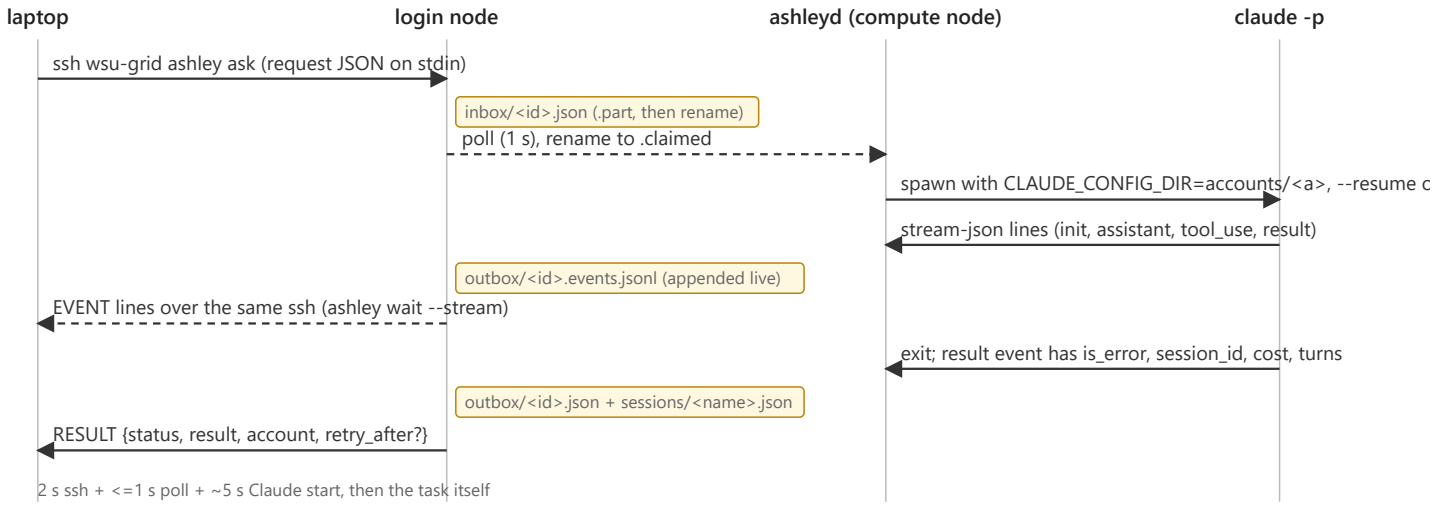
The three places and what runs in each; every laptop action is one ssh to the login node.

```
laptop                          warrior (login node)              compute node (Slurm job)
──────                          ────────────────────              ────────────────────────

VS Code — Ashley ext ──ssh──▶ ~/.ashley/bin/ashley ──files──▶  apptainer exec node22-bookworm
Claude Code — MCP ─┘           (bash: install, status,          └─ node ashleyd.js
Remote-SSH ──────────ProxyJump──────────────────────────────▶     └─ claude -p ... (2.1.283)
                               sbatch / squeue                     watches  workers/<w>/inbox
                                                                   writes   workers/<w>/outbox
                               shared home: ~/.ashley/{accounts,sessions,workers}
                               CephFS ROOT: containers/, npm/, workspaces/
```

The login node is a jump host. VS Code Remote-SSH connects to `voh3.nas.grid.wayne.edu` (any node where the student holds a job) through `ProxyJump wsu-grid`; C&IT forbids the VS Code server on `warrior` and kills it there. Compute nodes have internet, so Claude Code, `npm`, Hugging Face and Zenodo all work from inside the job.

The request queue



One request, from the laptop's ssh to the RESULT line.

A request is a JSON file. Its life:

1. The laptop writes `~/.ashley/workers/<w>/inbox/<id>.json` through `ashley send` (stdin, `.part` then rename, so the daemon never reads a half-written file).
2. `ashleyd.js` polls the inbox every second, claims the file by renaming it to `<id>.json.claimed` (an atomic operation on the shared filesystem; two daemons cannot both win).
3. It picks a credential (below), resolves the session, and spawns `claude -p <prompt> --output-format stream-json --verbose --permission-mode <mode> --max-turns N [--resume <sid> | --session-id <uuid>]` in the session's working directory, with `CLAUDE_CONFIG_DIR=~/.ashley/accounts/<account>`.

4. Every stream-json line is appended to `outbox/<id>.events.jsonl` as it arrives. `ashley wait --stream` tails that file to the laptop as `EVENT {…}` lines, so VS Code shows tool calls live.
5. On exit, the daemon writes `outbox/<id>.json` (`.part` then rename): status, result text, session id, account, cost, turns, duration, node, job. `ashley wait` prints it as `RESULT {…}`.

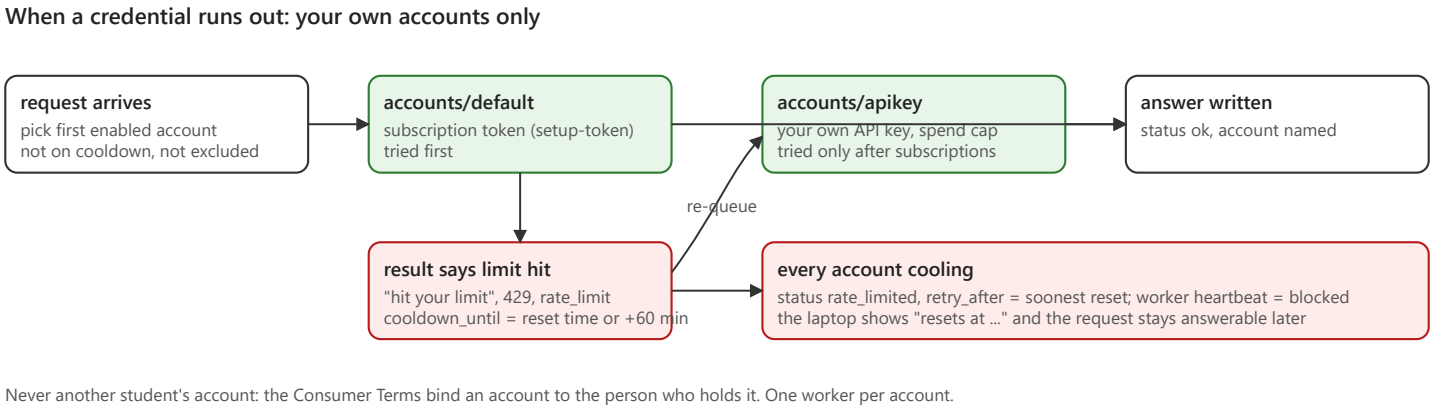
Latency is one ssh connection plus at most one poll interval plus Claude Code's own start-up (about 5 s cold on the grid). A queued request survives a daemon restart, a requeued job or a closed laptop: the daemon moves any `.claimed` file back to the inbox when it starts.

Why not a socket or a TCP port: the laptop can only reach the login node; the daemon runs on whichever node Slurm chose; the shared home is the one thing both see. Files are also what a student can debug with `ls` and `cat`.

Sessions

`~/.ashley/sessions/<name>.json` maps a name the student chose ("thesis", "figure-2") to a Claude Code session id, the working directory it is bound to and the account that owns it. The first turn of a session passes `--session-id <uuid>` so the id is known even if the run dies; later turns pass `--resume <uuid>`. Claude Code keeps the transcript under `<CLAUDE_CONFIG_DIR>/projects/<encoded cwd>/<uuid>.jsonl`, which is why a session stays bound to its directory.

Credentials



Rotation among a student's own credentials; a limit becomes a cooldown, not a failure.

Each `~/.ashley/accounts/<name>/` is both a credential and a `CLAUDE_CONFIG_DIR`, so accounts never share sessions, settings or caches:

- `token`: a long-lived subscription token from `claude setup-token` (the normal case). Exported to Claude Code as `CLAUDE_CODE_OAUTH_TOKEN`.

- `apikey`: an Anthropic API key (pay-per-use overflow). Exported as `ANTHROPIC_API_KEY`.
- `.credentials.json`: a copied interactive login. Works until its refresh token dies; not recommended (the probe on 2026-09-28 showed a copy rejected after eight idle days).

Order of use: subscriptions before API keys, `default` first. When a run ends with an error that matches the limit patterns in `protocol.ts` ("hit your limit", "rate_limit", "429", …), the account gets `state.json` with `cooldown_until` (the reset time Claude printed, else one hour), the request is re-queued for the next enabled account, and if none is left the result is written with `status: rate_limited` and `retry_after`. The worker's heartbeat reports `blocked` with `blocked_until` so the laptop can say "your window resets at 3 pm" instead of failing.

One person, one token. Pooling several people's subscriptions into one worker would breach the Consumer Terms (each account is for the person who holds it); the rotation exists for a student's own second credential, typically an API key with a spending cap.

Secrets: where they are and where they are not

Secret	Laptop	Grid	Never
GitHub, Hugging Face tokens	VS Code SecretStorage (OS keychain); injected into integrated terminals	`~/.ashley/secrets.env` (mode 600), read by the daemon into the worker's environment	job logs, `sbatch --export`, command lines
Claude subscription token	SecretStorage	`~/.ashley/accounts/default/token` (600)	laptop terminals
Anthropic API key	SecretStorage	`~/.ashley/accounts/apikey/apikey` (600)	laptop terminals (it would silently switch the local Claude Code to pay-per-use)
Grid password / OTP	nowhere	nowhere	typed once, in a terminal, to install the SSH key

Files travel over ssh **stdin** (`cat > file` with `umask 077`), never as arguments.

The container

`bootstrap.sbatch` (one job per user, 7 minutes measured) builds an unprivileged Apptainer **sandbox directory** from `docker://node:22-bookworm` on the student's CephFS space and installs `@anthropic-ai/claude-code@2.1.283` into a persistent npm prefix there. A sandbox rather than a `.sif`: SIF-to-sandbox conversion at exec time is what failed on this grid before, and several nodes lack `unsquashfs`. The host is CentOS

7 (glibc 2.17); Claude Code needs Node 22, which needs a newer glibc, hence the container (glibc 2.36 inside). Runtime order tried: `module load apptainer/1.2.2` (worked on `asx1`), the conda `singularity` 3.8.6, `module load singularity/3.5.2`, `/usr/bin/apptainer`. `~/.local/bin/singularity` is skipped on purpose.

The worker job (`worker.sbatch.tpl`) execs the daemon inside the sandbox with `$HOME`, the CephFS tree and a `/dev/shm` scratch bound to `/tmp` (`/tmp` on nodes is swept by a cleaner while jobs run). `--signal=B:TERM@120` gives the daemon two minutes to finish a turn at the wall limit; `--dependency=singleton` keeps one job per worker name; on the `requeue` QoS the job is `--requeue --open-mode=append` because the daemon is re-entrant.

The laptop side

- **Wizard** (`core/wizard.ts`): nine steps, each with `check()` and `run()`, over an abstract `WizardEnv` (settings, secrets, UI). The VS Code extension and the terminal CLI plug in their own UI. Rerunning does only what is missing.
- **SSH config** (`core/sshConfig.ts`): one managed block between markers, inserted first so it wins over any `Host *`. `MACs +hmac-sha1` because the grid's sshd offers nothing newer to Windows OpenSSH 9.5; `AddressFamily inet` because on Windows the IPv6 attempt to the login node times out first (13 s per connection measured, 2 s with IPv4 only); `IdentitiesOnly` so the agent's other keys are not tried; a wildcard `Host *.nas.grid.wayne.edu` with `ProxyJump` so no per-node edit is ever needed. On Windows the same block and key are mirrored into WSL.
- **Interactive steps** open a VS Code terminal whose *process* is `ssh` or `claude` (no shell, no quoting) and read its exit status.
- **Environment**: `environmentVariableCollection` (persistent) rather than editing `terminal.integrated.env` settings, so it can be cleared and does not leak into settings sync.
- **MCP**: `claude mcp add-json --scope user` with the server run by system `node` (>= 20) or by VS Code's own Electron with `ELECTRON_RUN_AS_NODE=1`, so students need not install Node. Fallback: `.mcp.json` in the workspace.

Xcode and other editors

`core/` never imports `vscode`. `dist/ashley-cli.js` runs the same wizard and the same grid calls from a terminal; `cli/ashley` is a ten-line ssh forwarder for machines without Node. The MCP server is editor-agnostic: any client that speaks MCP over stdio (Claude Code in Xcode's terminal, Claude Desktop, an Agent SDK script) can register it. A native Xcode extension is the wrong layer: Xcode Source Editor Extensions cannot open terminals, store secrets or run processes; the CLI plus MCP is the port.

What is deliberately not here

- No Nextflow yet. The queue and the worker are the substrate a pipeline can call (`ashley send` from a Nextflow process, `ashley result` in the next); the recipe's Nextflow section stays the plan.
- No shared lab account, no cross-student pooling.
- No X11. Remote-SSH renders images; `srun --x11` remains available for the rare tool that needs a display.

What you can ask, and what you cannot

Written for students. “The grid Claude” is the Claude Code worker running in your Slurm job with your subscription; “the laptop Claude” is the Claude Code extension in your VS Code.

Things that work well

Ask the grid Claude to do work where the data is. It runs in your grid home with your environment: read a dataset’s manifest, write a measurement script, run it on ten records, look at the numbers, fix the script, run it on all 802, write the results file, draft the JOURNAL line. Say which folder (a session is bound to one), which files, and what “done” looks like.

Ask for a Slurm job, not a long foreground run. “Write `slurm/measure.sh` from the lab template, show it to me, and after I say submit, submit it and record the job id.” The worker can submit and poll jobs; it should not itself run a two-hour computation in one turn.

Ask it to continue. Sessions are named. “In session `thesis`, what did we decide about the exclusion list?” picks up where it stopped, days later, on any node.

Ask the laptop Claude to delegate. “Check the grid status, then ask the grid Claude to run the pelvic width demo on ten records and tell me the mean.” The laptop Claude uses the `ashley_*` tools; the heavy part happens on the grid; the answer returns to your panel.

Long tasks. `ashley_send` queues; `ashley_result` collects later. Close the laptop. The job runs until its wall time (24 h by default, 30 days maximum on the `primary` QoS).

Images. Open your grid files with **Ashley: Open my grid files on a compute node**; click a PNG. No X server.

GPUs. Start a worker with one GPU when the task trains or infers; the grid has H200, L40S, A30 and V100-class nodes. Ask for a GPU only for that worker; CPU workers start sooner.

Things that do not work, and why

You cannot	Because
Run the worker or VS Code's server on the login node	C&IT kills processes on `warrior`; Ashley only ever connects VS Code to a compute node and runs a half-second file poll on the login node
Use someone else's Claude account, or pool accounts across students	Anthropic's consumer terms: an account is for the person who holds it. Ashley rotates only *your* credentials (your subscription, then your own API key)
Get more than your plan's window	A Pro/Max window is per account. When it is exhausted the worker reports `rate_limited` with the reset time and waits; an API key with a spending cap is the only overflow
Put patient data, PHI, survey CSVs or contact lists on the grid	The lab's IRB umbrella (HPR 2026-269) covers public, de-identified datasets only; anything else needs its own filing first (RECIPE.md, data rules)
Push the grid Claude to `--dangerously-skip-permissions` for everything	It is available (`mode: bypassPermissions`) but the default `acceptEdits` lets it edit files and run tools while refusing destructive commands; use bypass only inside a project folder you can `git checkout` back
Ask for interactive programs (ITK-SNAP, a GUI) through the worker	The worker is headless. Run GUI tools locally on rsynced results, or with `srun --x11` in your own shell
Expect a reply in under a few seconds	One ssh connection (2 s), the poll (1 s) and Claude Code's start (5 s) come before the first token; a real task takes minutes
Keep a worker forever	Jobs have wall limits; the `requeue` QoS may requeue the job (it resumes). Start another; queued requests wait
Store 200 GB in your home	Home is a 4 TB soft quota shared with everything else; Ashley's container and workspaces live on the lab's CephFS volume when you have access, and the doctor shows the quota

What the worker Claude can and cannot see

Can: your grid home and CephFS space, the internet (PubMed, Hugging Face, GitHub, Zenodo, pip), your GitHub and Hugging Face tokens (as environment variables, for `git push` and `hf download`), the Slurm commands, the lab's Python environments.

Cannot: your laptop's files (it is on the grid), your Claude password (it holds a token you can revoke), other students' homes, the login node's CPUs for anything heavy.

Good prompts

- "In session `pelvis`, working in `~/Student_Projects/projects/lee_pelvic_width`, read `proposal.md`, then write `code/measure_width.py` that measures the bi-iliac width on the label volumes listed in `lists/dev10.txt`, run it, and show me the ten numbers with units."
- "Write the Slurm script for the full 802 from the lab template. Do not submit; show it."
- "Compare the distribution in `results/widths.json` with the published values in `docs/normative.md`; flag anything outside the widest published SD."
- "Append today's entry to JOURNAL.md: what ran, job ids, what came out, what is next."

Prompts that will disappoint

- "Do my whole project." (No question, no data, no bar for failure. Start with PREREG.md.)
- "Make the number come out significant." (The recipe forbids editing results by hand and so does the worker's default mode; fix the script, rerun, report what you get.)
- "Download the CTs." (195 GB into a shared quota, on a request that may be repeated. Ask your lead for the shared path first.)

Installing and building

For a student (two minutes)

1. Install VS Code and, from the Extensions view, **Claude Code** (`anthropic.claude-code`) and **Remote - SSH** (`ms-vscode-remote.remote-ssh`). Sign in to Claude Code with your Claude Pro or Max account.
2. Get the file `ashley-wsu-1.0.0.vsix` from your lead (or the repository's `dist/` folder).
3. In VS Code: Extensions view, the `...` menu at the top right, **Install from VSIX...**, pick the file. Or from a terminal: `code --install-extension ashley-wsu-1.0.0.vsix`.
4. Command Palette (Ctrl/Cmd+Shift+P): **Ashley: Set up my research environment (wizard)**.

Before the wizard you need, once: a grid account (request it at tech.wayne.edu/kb/high-performance-computing naming your sponsor), and Google Authenticator set up through ondemand.grid.wayne.edu, Interactive Apps, 2FA Setup (VPN required for OnDemand).

What the wizard writes on your laptop

Where	What
`~/.ssh/id_ed25519_wsu`, `.pub`	a key pair without passphrase
`~/.ssh/config`	one block between `# >>> ashley` and `# <<< ashley` (a backup `config.ashley.bak` is kept)
OS keychain (Windows Credential Manager / macOS Keychain / libsecret)	GitHub, Hugging Face, Claude tokens
VS Code user settings `ashley.*`	your AccessID and preferences
WSL (Windows only, optional)	the same key and block in the WSL home

On the grid it writes `~/.ashley/` (scripts, tokens with mode 600, workers, sessions), a block in `~/.bashrc`, and large files under `/rs/rs_grp_oschome/<AccessID>/ashley` or `~/ashley`.

To undo: remove the block from `~/.ssh/config`, delete the key, run **Ashley: Manage stored tokens** to delete tokens, uninstall the extension; on the grid, `rm -rf ~/.ashley` and the block in `~/.bashrc`.

Building from source

Requirements: Node.js 22 (the packaging tool refuses older), Git.

```
git clone <this repository> ashley
cd ashley/extension
npm install
npm run build          # bundles dist/extension.js, dist/mcp-server.js, dist/ashley-cli.js,
         copies ../grid into dist/grid
npm run test:unit      # 22 tests: ssh config, protocol, slurm parsing, and the daemon with a
         fake claude
npm run test:vscode    # downloads a VS Code build once, runs the integration suite
npm run package        # writes ../dist/ashley-wsu-<version>.vsix
code --install-extension ../dist/ashley-wsu-1.0.0.vsix
```

On a Windows machine without Node on `PATH`, the portable build works: download `node-v22.x-win-x64.zip` from nodejs.org, unzip, and prefix `PATH` with that folder for the commands above.

The extension bundles everything it needs (`vsce package --no-dependencies` plus esbuild), so the `.vsix` has no `node_modules`; students never run `npm`.

The terminal CLI (macOS, Linux, Xcode users)

`extension/dist/ashley-cli.js` is the wizard and the grid calls without VS Code:

```
node dist/ashley-cli.js setup            # the same nine steps, in the terminal
node dist/ashley-cli.js doctor
node dist/ashley-cli.js worker start --name default --hours 24
node dist/ashley-cli.js ask --session thesis "read PREREG.md and list its hypotheses"
```

`cli/ashley` (bash) and `cli/ashley.ps1` forward any command to the grid over ssh and need only an ssh client and the config block (`node dist/ashley-cli.js ssh-config` prints it).

Updating

Install the new `.vsix` the same way; run **Ashley: Install or update Ashley on the grid** to push new grid scripts (a job rebuilds the container only if the pinned Claude Code version changed).

Troubleshooting

Run **Ashley: Check everything (doctor)** first. It prints one line per check on the laptop and, when the connection works, the grid's own checks.

Symptom	Cause	Fix
`no matching MAC found. Their offer: hmac-sha1`	your `ssh` config lacks the Ashley block	rerun the wizard (step 3 rewrites the block with `MACs +hmac-sha1`)
Every grid action takes 13 s	IPv6 attempt on Windows	the block sets `AddressFamily inet`; rerun the wizard if you edited the file
Wizard step 4 loops on the password	wrong password/code, or the account is not yet created	try in a terminal: `ssh ab1234@warrior.grid.wayne.edu`
`Permission denied (publickey...)` after step 4	the key was not appended to `authorized_keys`	rerun step 4; on the grid, `cat ~/.ssh/authorized_keys` should contain “ashley”
Ashley view: *Grid installing* for more than 30 min	the bootstrap job is queued	`ssh wsu-grid squeue -u $USER`; if it pends for long, `ssh wsu-grid "~/.ashley/bin/ashley bootstrap --resubmit --partition secp --qos secondary"`
Worker shows PENDING for long	the `requeue` QoS pool is busy	start it with the `secp` / `secondary` settings (`ashley.worker.partition`, `ashley.worker.qos`)
Ask returns `no enabled Claude account`	step 7 was skipped	**Ashley: Store or replace my Claude token for the grid**
Ask returns `Failed to authenticate`	the token expired or was revoked	run `claude setup-token` again and store the new token
Ask returns `rate_limited` with a time	your subscription window is exhausted	wait for the time shown, or add an API key (step 8)
Remote-SSH: *Connecting with SSH timed out*	the compute node’s sshd needs about 25 s per session; the first connection also downloads the server	Ashley sets `remote.SSH.connectTimeout` to 60; raise it to 300 in Settings, connect again
Remote-SSH: *glibc* or *exit code 207*	the C&IT sysroot variables are not in `~/.bashrc`, or a `return` guard precedes them	`ssh wsu-grid "~/.ashley/bin/ashley install"` rewrites the block; move it above any `[ -z "$PS1" ] && return`
VS Code connects to `warrior` instead of a node	a job was not running	start a worker; Ashley connects only to nodes where you hold a job

Symptom	Cause	Fix
`home quota` above 90 %	your home is full	large files belong on the lab volume; ask your lead for the path; `ashley.gridRoot`
The MCP tools do not appear in Claude Code	the bridge was not registered, or Claude Code was open before registration	**Ashley: Connect the Claude Code extension to the grid**, then start a new conversation

Logs

- Laptop: *View, Output, Ashley* (every command and its exit code).
- Grid: `~/.ashley/log/bootstrap_<job>.out`, `~/.ashley/workers/<name>/log/ashleyd.log`, `~/.ashley/workers/<name>/log/slurm_<job>.out`, and every request's `outbox/<id>.events.jsonl`.
- Remote-SSH: *View, Output, Remote - SSH*.

Undo

Laptop: remove the block from `~/.ssh/config` (a `config.ashley.bak` is kept), delete `~/.ssh/id_ed25519_wsu*`, **Ashley: Manage stored tokens** to delete tokens, uninstall the extension. Grid: `ashley worker stop`, `rm -rf ~/.ashley`, remove the block from `~/.bashrc`, delete the container folder under your storage root.

Appendix

- `docs/RESEARCH.md`: the research behind the design (Claude Code headless mode, subscription limits and terms, containers on this grid, multi-account tools, orchestration patterns, VS Code Remote-SSH on old glibc, Xcode, literature APIs), with sources and which findings were independently verified.
- `docs/CHECKPOINT_2026-09-28.md`: the state of the project on the day this manual was built, including what was tested and what was not.
- `Student_Projects/RECIPE.md`: the lab's method; Ashley is the on-ramp to it.
- C&IT: "How to SSH to the Grid in Visual Studio Code" and "How do I set up and use Google Authenticator for the Grid" (services.wayne.edu knowledge base).